Rails Model Support

In general, we want our web applications to keep their information in a relational database. Order-entry systems will store orders, line items, and customer details in database tables. Even applications that normally use unstructured text, such as weblogs and news sites, often use databases as their back-end data store.

Although it might not be immediately apparent from the database queries you've seen so far, relational databases are designed around mathematical set theory. This is good from a conceptual point of view, but it makes it difficult to combine relational databases with object-oriented (OO) programming languages. Objects are all about data and operations, and databases are all about sets of values. Operations that are easy to express in relational terms are sometimes difficult to code in an OO system. The reverse is also true.

Over time, folks have worked out ways of reconciling the relational and OO views of their corporate data. Let's look at the way that Rails chooses to map relational data onto objects.

Object-Relational Mapping

Object-relational mapping (ORM) libraries map database tables to classes. If a database has a table called `orders`, our program will have a class named `Order`. Rows in this table correspond to objects of the class—a particular order is represented as an object of the `Order` class. Within that object, attributes are used to get and set the individual columns. Our `Order` object has methods to get and set the amount, the sales tax, and so on.

In addition, the Rails classes that wrap our database tables provide a set of class-level methods that perform table-level operations. For example, we might need to find the order with a particular ID. This is implemented as a class method that returns the corresponding `Order` object. In Ruby code, that might look like this:

```
order = Order.find(1)
puts "Customer #{order.customer_id}, amount=$#{order.amount}"
```